you're not touching the actual database. Using the database is, for testing purposes, slow. Very slow. Using test doubles to stub database access is one strategy for avoiding the database for the purpose of speeding up tests. In the meantime, remember that this stub shouldn't be used to verify that the find method works; it should be used by other tests that need the find method along the way to the logic that is under test. For instance, this can be used to ensure that a controller that is accessing the database gets an instance that is controlled by the test.

That said, in practice you also should avoid stubbing the find method because it's part of an external library—you might be better off creating a model method that has a more meaningful and specific behavior and stubbing that method.

On a related note, if you wanted to create multiple partial stubs from the same class and have them all behave the same way, you could do so with the method allow_any_instance_of, as you see here:

```
allow_any_instance_of(Project).to receive(:save).and_return(false)
```

This snippet causes any instance of Project that is created to return false when saved. In other words, it simulates a failure condition.

You should use allow_any_instance_of very sparingly, as it often implies you don't really understand what the underlying code is doing or where objects might come from. However, when you're testing legacy code you may genuinely not know where the object comes from, and allow_any_instance_of might be a lesser evil. Also, sometimes framework concerns in Rails make using allow_any_instance_of easier than managing the set of objects that might be returned by some distant method. In either case, though, consider the possibility of refactoring the code or test to avoid allow_any_instance_of. The RSpec docs explicitly recommend not using this feature if possible, since it is "the most complicated feature of rspec-mocks, and has historically received the most bug reports."

Prescription 20	The use of the allow_any_instance_of stub modifier often means the underlying code being tested could be refactored with a more useful method to stub.

A common use of stub objects is to simulate failure and exception conditions. If you want your stubbed method to raise an exception, you can use the and_raise method, which takes an exception class and an optional message:

```
allow(stubby).to receive(:user_count).and_raise(Exception, "oops")
```